

Cortex-M3 & FreeRTOS Setup Guide

A complete reference for VS Code + QEMU bare-metal debugging on Windows.

This document is a comprehensive, step-by-step guide to reproducing the Cortex-M3 emulator environment using QEMU, the Arm GNU Toolchain, and VS Code. It also includes instructions for integrating and debugging FreeRTOS.

Phase 1: Toolchain and Environment Setup

Step 1: Install the Arm GNU Toolchain

We need the cross-compiler for Arm (specifically the `arm-none-eabi` toolchain) to compile C code for the virtual Cortex-M3 chip. It also provides the GDB debugger.

- Download the Windows installer from the Arm GNU Toolchain page. Look for the **AArch32 bare-metal target (arm-none-eabi)**, usually named something like `arm-gnu-toolchain-13.2.Rel1-mingw-w64-i686-arm-none-eabi.exe`.
- Run the installer. **CRUCIAL:** On the final step, check the box that says "Add path to environment variable".
- Verify by opening a new Command Prompt and typing: `arm-none-eabi-gcc --version`.

Step 2: Install QEMU for Windows

QEMU acts as the virtual hardware, emulating an LM3S6965 Evaluation Board (which uses a Cortex-M3).

- Download the QEMU binaries for Windows from qemu.weilnetz.de/w64/.
- Install using the default settings (usually to `C:\Program Files\qemu`).
- **Manually add to System Path:** Open Windows Environment Variables -> Edit System Variables -> find `Path` -> Add a New entry for `C:\Program Files\qemu`.
- Verify by opening a new terminal and typing: `qemu-system-arm --version`.

Step 3: VS Code Extensions

Install these two extensions in VS Code:

- **C/C++ (by Microsoft):** For syntax highlighting and IntelliSense.
- **Cortex-Debug (by marus25):** For connecting VS Code's debugger UI to GDB and QEMU.

Phase 2: Bare-Metal Setup

Step 4: The Linker Script (`linker.ld`)

The linker script tells the compiler where to place the code (FLASH) and variables (RAM). We use a comprehensive version that sets up the boundaries for the C Standard Library (needed later for FreeRTOS memory functions).

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 256K
    RAM (rwx)  : ORIGIN = 0x20000000, LENGTH = 64K
}

SECTIONS
{
    .text :
    {
        KEEP(*(.isr_vector))
        *(.text*)
        *(.rodata*)
    } > FLASH

    .data :
    {
        *(.data*)
    } > RAM

    .bss :
    {
        __bss_start__ = .;
        *(.bss*)
        *(COMMON)
        __bss_end__ = .;
    } > RAM

    PROVIDE(end = .);
}
```

Phase 3: FreeRTOS Integration

Step 5: Copy FreeRTOS Files

Download the **FreeRTOS 202604.00 LTS** source code. In your project, create a **FreeRTOS** folder and copy only the necessary files:

- From **FreeRTOS-Kernel**: Copy **list.c**, **queue.c**, **tasks.c**, and **timers.c**.
- Copy the entire **include** folder.
- From **portable/GCC/ARM_CM3**: Copy **port.c** and **portmacro.h**.
- From **portable/MemMang**: Copy **heap_1.c**.

Step 6: FreeRTOS Configuration (**FreeRTOSConfig.h**)

Create this file in your project root. It handles clock speed and interrupt mapping.

```

#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

#define configUSE_PREEMPTION 1
#define configUSE_TICKLESS_IDLE 0
#define configCPU_CLOCK_HZ ( 50000000UL )
#define configTICK_RATE_HZ ( 1000 )
#define configMAX_PRIORITIES 5
#define configMINIMAL_STACK_SIZE 128
#define configMAX_TASK_NAME_LEN 16
#define configUSE_16_BIT_TICKS 0
#define configIDLE_SHOULD_YIELD 1
#define configUSE_TASK_NOTIFICATIONS 1
#define configUSE_TIME_SLICING 1
#define configSUPPORT_STATIC_ALLOCATION 0
#define configSUPPORT_DYNAMIC_ALLOCATION 1
#define configTOTAL_HEAP_SIZE ( ( size_t ) ( 10 * 1024 ) )

/* Cortex-M3 specific interrupt routing */
#define configPRIO_BITS 3
#define configLIBRARY_LOWEST_INTERRUPT_PRIORITY 15
#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5
#define configKERNEL_INTERRUPT_PRIORITY
( configLIBRARY_LOWEST_INTERRUPT_PRIORITY << ( 8 - configPRIO_BITS ) )
#define configMAX_SYSCALL_INTERRUPT_PRIORITY
( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << ( 8 - configPRIO_BITS ) )

/* Map FreeRTOS handlers to ARM Vector Table */
#define vPortSVCHandler SVC_Handler
#define xPortPendSVHandler PendSV_Handler
#define xPortSysTickHandler SysTick_Handler

/* Enable API Functions */
#define INCLUDE_vTaskPrioritySet 1
#define INCLUDE_uxTaskPriorityGet 1
#define INCLUDE_vTaskDelete 1
#define INCLUDE_vTaskSuspend 1
#define INCLUDE_vTaskDelayUntil 1
#define INCLUDE_vTaskDelay 1

#endif /* FREERTOS_CONFIG_H */

```

Step 7: Application Code (`main.c`)

This includes the full Vector Table and two test tasks.

```

#include "FreeRTOS.h"
#include "task.h"

volatile int task1_counter = 0;
volatile int task2_counter = 0;

void Task1(void *pvParameters) {
    while(1) {
        task1_counter++;
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

void Task2(void *pvParameters) {
    while(1) {
        task2_counter++;
        vTaskDelay(pdMS_TO_TICKS(20));
    }
}

void main(void) {
    xTaskCreate(Task1, "Task1", configMINIMAL_STACK_SIZE, NULL, 1, NULL);
    xTaskCreate(Task2, "Task2", configMINIMAL_STACK_SIZE, NULL, 1, NULL);
    vTaskStartScheduler();
    while(1);
}

void Reset_Handler(void) { main(); }
void Default_Handler(void) { while(1); }

extern void SVC_Handler(void);
extern void PendSV_Handler(void);
extern void SysTick_Handler(void);

__attribute__((section(".isr_vector")))
void (*const vector_table[])(void) = {
    (void (*)(void))0x20010000, Reset_Handler, Default_Handler,
    Default_Handler, Default_Handler, Default_Handler, Default_Handler,
    0, 0, 0, 0, SVC_Handler, Default_Handler, 0, PendSV_Handler,
    SysTick_Handler
};

```

Step 8: The Compilation Command

Run this command in the VS Code terminal. Note the use of `--specs=nosys.specs` which safely provides standard C functions (like `memcpy`) required by FreeRTOS on bare-metal systems.

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -T linker.ld -g -O0 --specs=nosys.specs -I.
-I./FreeRTOS/include -I./FreeRTOS main.c FreeRTOS/list.c FreeRTOS/queue.c FreeRTOS/
tasks.c FreeRTOS/timers.c FreeRTOS/port.c FreeRTOS/heap_1.c -o firmware.elf
```

Note: Ignore any warnings regarding `_close`, `_read`, etc. not being implemented. This is expected and safe on bare-metal.

Phase 4: Debugging Configurations

Step 9: Launch Configuration (`launch.json`)

Create a `.vscode` folder in your project root, and add a `launch.json` file with this content:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "QEMU FreeRTOS Debug",
      "type": "cortex-debug",
      "request": "launch",
      "servertype": "qemu",
      "executable": "${workspaceFolder}/firmware.elf",
      "cpu": "cortex-m3",
      "machine": "lm3s6965evb"
    }
  ]
}
```

Important Note on QEMU and RTOS Thread Awareness:

Normally, the `cortex-debug` extension accepts an `"rtos": "FreeRTOS"` flag to show thread lists in the UI. However, **QEMU's built-in GDB server does not support RTOS awareness**. Including the flag will crash the debug session.

Removing the flag allows standard debugging. The FreeRTOS scheduler operates perfectly in QEMU, and you can verify context switching by placing breakpoints in different tasks and watching execution jump between them.

Learning Roadmap

With this sandbox setup, you can learn and test the following FreeRTOS features entirely through QEMU:

Concept	How to Test
Queues	Use <code>xQueueCreate</code> and <code>xQueueSend/Receive</code> to pass sensor data between tasks safely.
Mutexes	Use <code>xSemaphoreCreateMutex</code> to protect a global array from being accessed by two tasks simultaneously.
Semaphores	Use a Binary Semaphore to pause a task until another task signals it to wake up.
Timers	Use <code>xTimerCreate</code> for software-managed callbacks independent of task loops.
Heap Mgmt	Switch to <code>heap_4.c</code> and monitor RAM usage using <code>xPortGetFreeHeapSize()</code> .